LUNA

Menstrual Cycle Tracker and Wellness App

PROJECT REPORT

Student Name: S.KAMALINI

Register Number: 2025BCS0169

Course / Workshop: IT Workshop III

Department: Computer Science and Engineering

Institution: IIIT Kottayam

Submission Date: 26-10-2026

Abstract

Luna is a Python-based menstrual cycle tracking and wellness application. It is designed to allow users to record period dates, maintain daily mood and symptom notes, review cycle history, and explore summaries of recorded information. The application uses Streamlit for its interactive interface and SQLite for data storage. Pandas and NumPy support data processing and statistical calculations, while Matplotlib is used to create visualisations. The application estimates a possible next period date from previously recorded cycle lengths and provides an in-app reminder. The estimate is informational only because cycle timing varies, and the application is not intended for medical diagnosis or treatment.

1. Introduction

Keeping track of period dates and related daily experiences can help a person review their own records over time. When information is scattered across notes or calendars, it may be difficult to calculate the time between periods or recognise what has been recorded on particular days. Luna brings period records and daily wellness entries into one application and presents basic summaries and charts.

1.1 Problem Statement

A user needs a simple way to record menstrual dates and daily wellness observations, review the history, and calculate an approximate next period date from available records. The application should also provide data export and deletion options so that users can manage their stored information.

1.2 Objectives

To record period start and end dates.

To estimate cycle length and a possible upcoming period date from previous records.

To allow users to record daily mood, energy, symptoms, and notes.

To summarise and visualise recorded cycle and wellness information.

To store records using SQLite.

To provide CSV export and an option to delete stored records.

To demonstrate Python programming, database operations, data analysis, and visualisation.

2. Technologies Used

Python: Application logic, calculations, and data processing.

Streamlit: Interactive application interface and page navigation.

SQLite: Storage of period records and daily journal entries.

Pandas: Tabular data handling, filtering, and summaries.

NumPy: Numerical calculations such as mean and standard deviation.

Matplotlib: Charts showing cycle lengths, mood, symptoms, and energy levels.

Jupyter Notebook: Demonstration and documentation of data analysis and database operations.

3. System Design and Methodology

The application is organised into separate Python modules so that the interface, database operations, cycle calculations, analytics, and reminder messages can be maintained independently.

3.1 Main Components

app.py: Contains the Streamlit interface, navigation, forms, and display of information.

database.py: Handles SQLite table creation and functions for adding, retrieving, validating, deleting, and exporting records.

cycle.py: Calculates cycle lengths and estimates the next period date using saved period start dates.

analytics.py: Handles data summaries and visualisations used by the application.

reminders.py: Displays an in-app reminder based on the current period estimate.

3.2 Data Flow

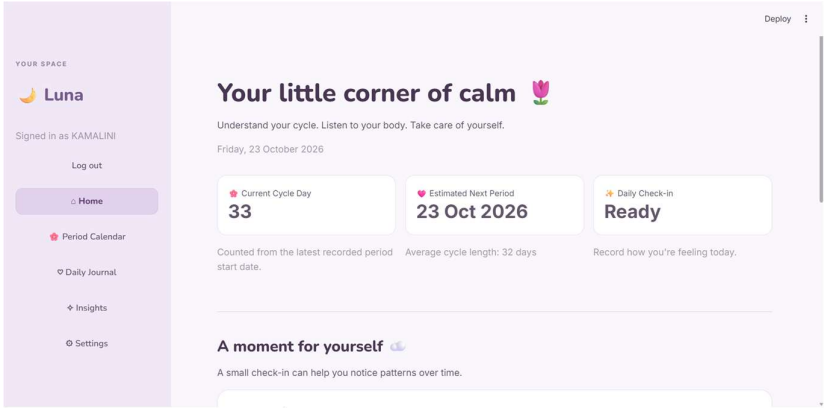
The user enters information through a Streamlit form. The application validates the input before saving it to the SQLite database. When a page needs to display history or insights, the application retrieves the relevant records, processes them, and displays tables, metrics, or charts. Export functionality creates CSV files from the stored records.

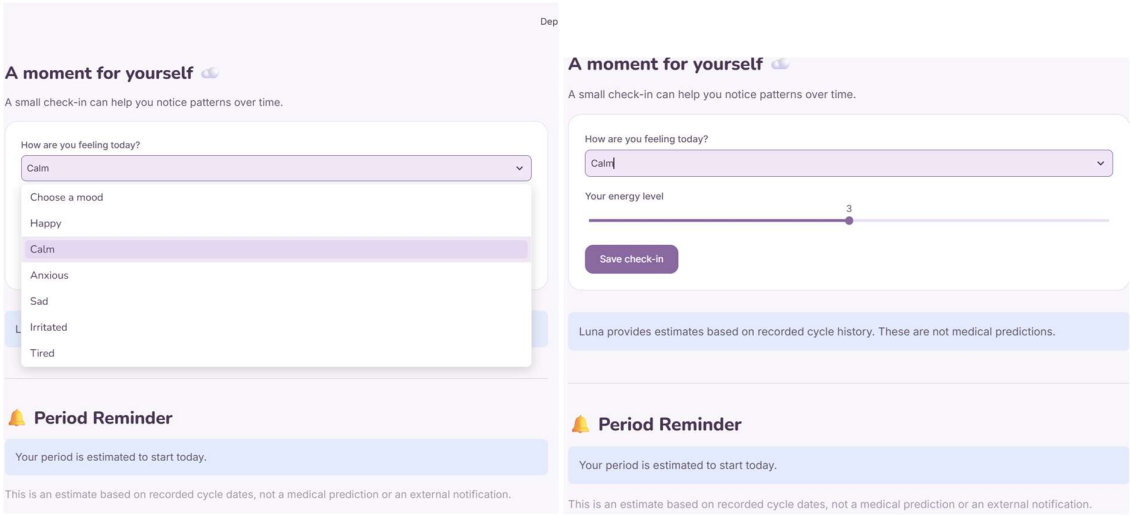
The Jupyter Notebook separately demonstrates analysis using sample data and an in-memory SQLite database. These demonstration records are not automatically added to the application's database.

4. Application Features

4.1 Home

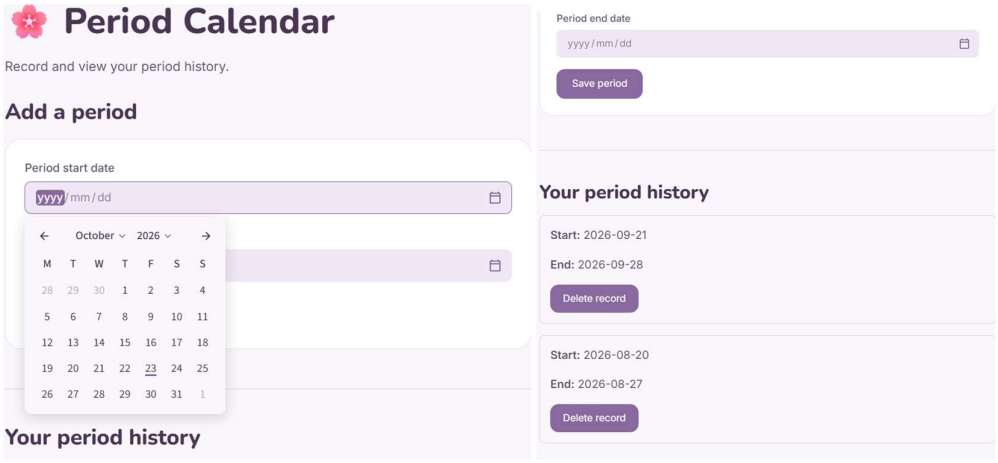
The Home page summarises useful information, such as the current cycle day when it can be calculated, the estimated next period date, and an in-app reminder message. The estimate depends on the dates stored by the user.





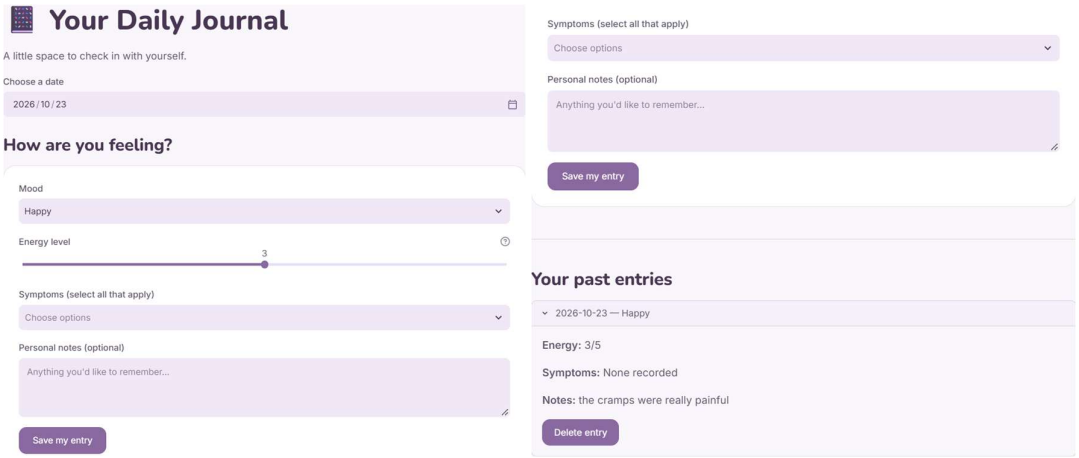
4.2 Period Calendar

The Period Calendar allows users to enter period start and end dates and review saved records. Input validation is designed to reject invalid date ranges, future dates, excessively long durations, duplicate records, and overlapping records.



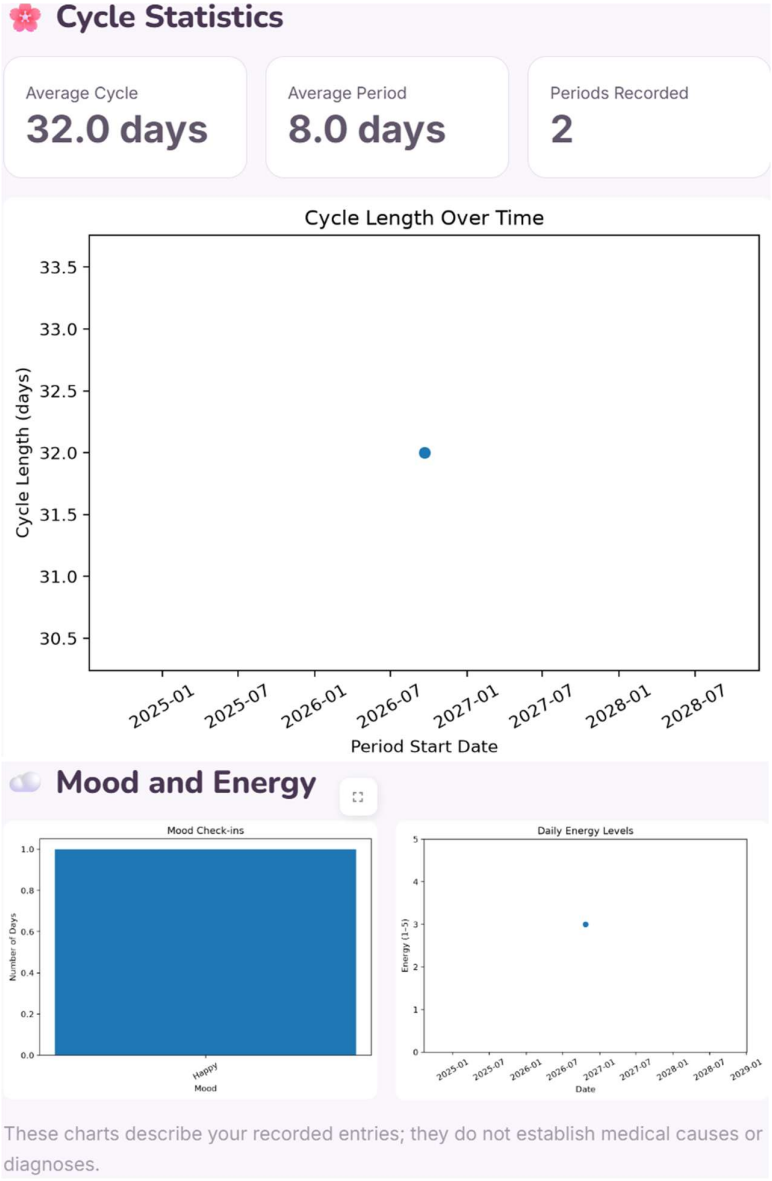
4.3 Daily Journal

The Daily Journal allows users to record their mood, energy level, symptoms, and optional notes for a selected date. It is designed to prevent future-dated entries and update an existing entry for the same date rather than create unwanted duplicates.



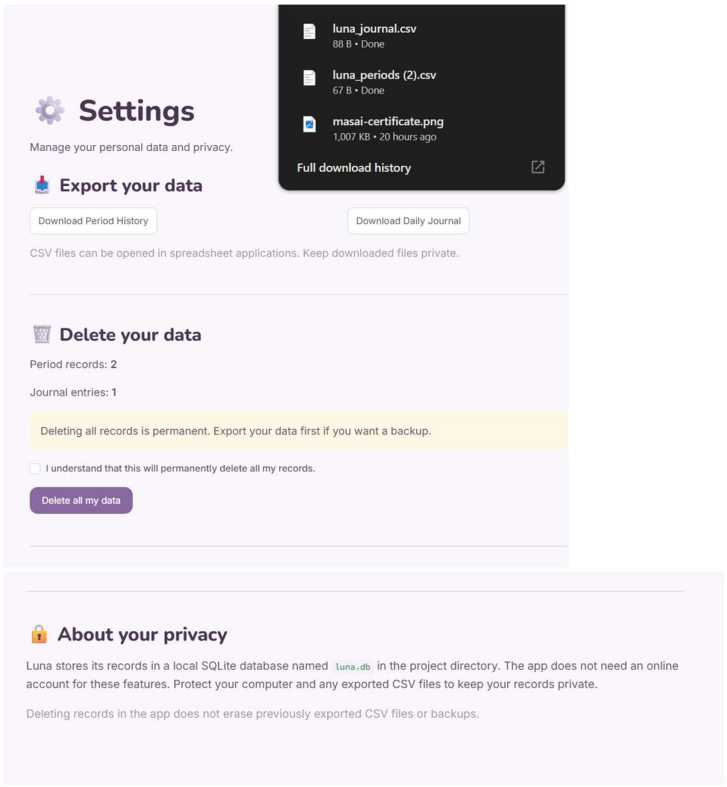
4.4 Insights

The Insights page presents summaries of stored records and charts that help users review cycle lengths and daily wellness entries. The notebook includes separate examples of calculations and visualisations using Pandas, NumPy, and Matplotlib.



4.5 Settings and Data Management

The Settings page provides CSV export for period and journal data, displays record counts, and offers an option to delete all records after confirmation. Since records may contain personal information, exported files should be stored securely. Deletion should be used carefully because it removes saved records.



5. Data Analysis and Calculations

Cycle length is calculated as the number of days between the start of one period and the start of the next period. Period duration is calculated as the difference between the end and start dates plus one, so that both the first and last days are included.

The notebook uses example period start dates of 1 June, 29 June, 27 July, and 25 August 2026. These produce cycle lengths of 28, 28, and 29 days.

Average cycle length: 28.33 days

Shortest recorded cycle: 28 days

Longest recorded cycle: 29 days

Standard deviation: Approximately 0.47 days

These values describe only the example dataset and should not be treated as conclusions about an individual user's cycle.

Daily journal analysis uses Pandas to count mood and symptom categories and calculate the average energy level. Matplotlib bar charts show mood and symptom frequencies, while a line chart displays recorded energy levels over time. These charts demonstrate how data can be explored, but they do not establish medical relationships.

5.1 Next Period Estimate

The application calculates the intervals between recorded period start dates and uses a recent average to estimate the next start date. An estimate can be inaccurate when cycle lengths change, when there are too few records, or when records are incomplete. Therefore, the estimated date is not a guaranteed start date.

6. Testing and Results

The following test cases can be used to evaluate the application. The observed result should be completed after testing each feature in the actual running application.

Test case	Expected result	Observed result
Add a valid period	The record is saved and appears in the history.	To be completed after testing
Enter an end date before the start date	A validation message appears, and the record is not saved.	To be completed after testing
Add an overlapping period	The record is rejected with a useful message.	To be completed after testing
Save a daily journal entry	The entry appears in the journal.	To be completed after testing
Export records	CSV files are generated with saved data.	To be completed after testing
Delete all records with confirmation	Records are deleted only after confirmation.	To be completed after testing
Register and log in (if implemented)	The account can be created and accessed.	To be completed only if this feature is present
Check account separation (if implemented)	Each account sees only its own records.	To be completed only after verification

Before submission, replace the placeholder results with the outcomes actually observed. Add genuine screenshots of the running Home, Period Calendar, Daily Journal, Insights, and Settings pages if required by the instructor.

7. Limitations

The next period date is an estimate and cannot guarantee when a period will begin.

The reminder is displayed inside the application; it is not a push notification when the application is closed.

Insights are limited by the amount and quality of data entered by the user.

The notebook's sample records are for demonstration and are separate from the application's persistent records.

The application is an educational personal-tracking tool and does not provide medical advice.

8. Future Enhancements

Add optional configurable reminder settings and scheduled email reminders.

Provide richer filters and date-range summaries.

Improve chart presentation and downloadable summary reports.

Add database backup and restore options.

Provide clearer privacy guidance and stronger protection for exported files.

If account login is implemented, test account-specific data separation and provide a secure account recovery process.

9. Conclusion

Luna brings period-date recording and daily wellness journaling into a single Python application. It demonstrates the use of Streamlit for an interactive interface, SQLite for data storage, and Pandas, NumPy, and Matplotlib for analysis and visualisation. It also demonstrates input validation, data export, and record deletion. The project provides a practical example of integrating multiple Python tools into one application while recognising that cycle estimates are approximate and are not a substitute for professional medical advice.

References

Python Software Foundation. Python Documentation. https://docs.python.org/3/

Streamlit Documentation: https://docs.streamlit.io/

SQLite Documentation: https://www.sqlite.org/docs.html

Pandas Documentation: https://pandas.pydata.org/docs/

NumPy Documentation: https://numpy.org/doc/

Matplotlib Documentation: https://matplotlib.org/stable/